

Internet of Things

Lab 3



Ultrasonic Sensor

Ultrasonic sensor → is a device that uses ultrasonic sound waves to detect the presence of objects and measure distances.

- ❑ distance measuring sensor which has a range from 2cm to 400cm.
- ❑ The sensor is composed of two ultrasonic transducers. One is a transmitter that outputs ultrasonic sound pulses and the other is a receiver that listens for reflected waves



Ultrasonic Sensor

How it measures the distance?

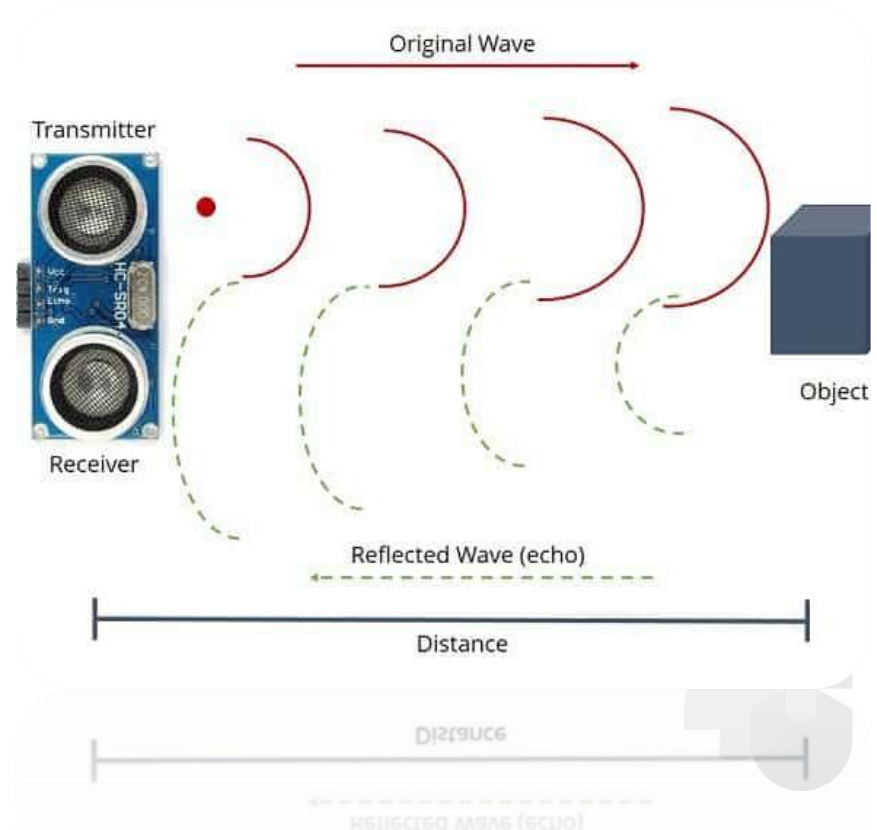
1. The sensor emits a high-frequency sound wave (ultrasonic wave) towards the target object.
2. The sound wave travels through the air until it reaches the object and is then reflected towards the sensor.
3. The sensor detects the reflected sound wave and measures the time it takes for the wave to return to the sensor.
4. By knowing the speed of sound in the air and the time taken for the wave to return, the sensor can calculate the distance to the object using the formula:

❖ **Distance = Speed of Sound * Time.**

Note: divide the time by 2 because the sound wave travels to the object and back to the sensor.

Speed of Sound = 340m/s.

Distance = Speed of Sound * Time/2.



Ultrasonic Sensor

- 1. The microcontroller sends a short pulse to the Trig pin of the ultrasonic sensor to trigger the sensor to emit an ultrasonic pulse.
- 2. The ultrasonic sensor transmitter emits an ultrasonic sound wave that travels through the air.
- 3. The ultrasonic wave hits an object and gets reflected back towards the sensor.
- 4. The sensor's receiver detects the reflected wave and sends a high signal to the Echo pin.
- 5. The Echo pin of the sensor will remain high as long as the reflected wave is being received by the sensor.
- 6. The duration for which the Echo pin remains high corresponds to the time taken for the ultrasonic wave to travel to the object and back.
- 7. Using the speed of sound in air and the time measured, the microcontroller calculates the distance to the object. $\rightarrow \text{distance} = \text{speed of sound} * \text{time calculated} / 2$.

Convert from m/s to cm/ μ s $\rightarrow$ speed = .034cm/ μ s.

distance = .034*time(calculated by pulseIn() built-in function) /2



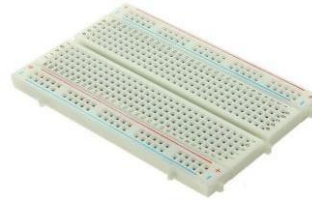
Project Component we need !

1 ESP-32S 30Pin Development Board WiFi Bluetooth.



ESP32S - 30PIN

2 breadboard.



3 Ultrasonic Sensor (HC-SR04).

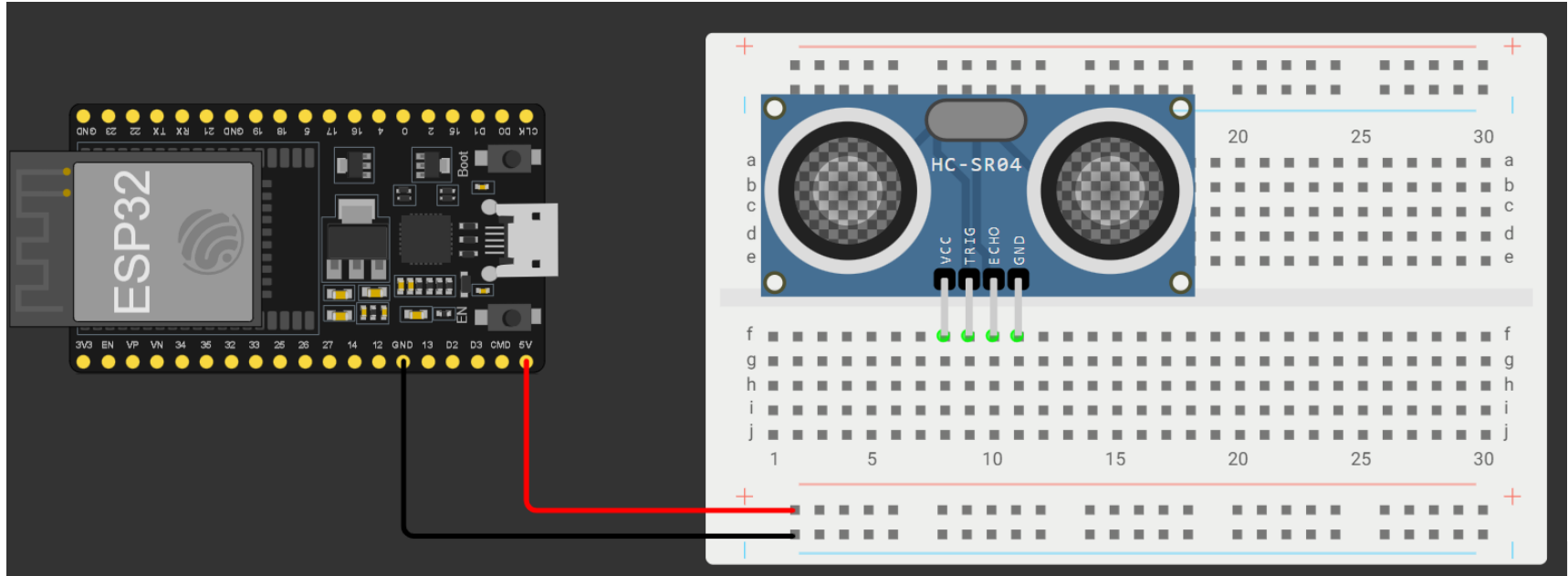


4 Wires



Ultrasonic Sensor

Step1:Connecting the Ultrasonic Sensor to the Breadboard



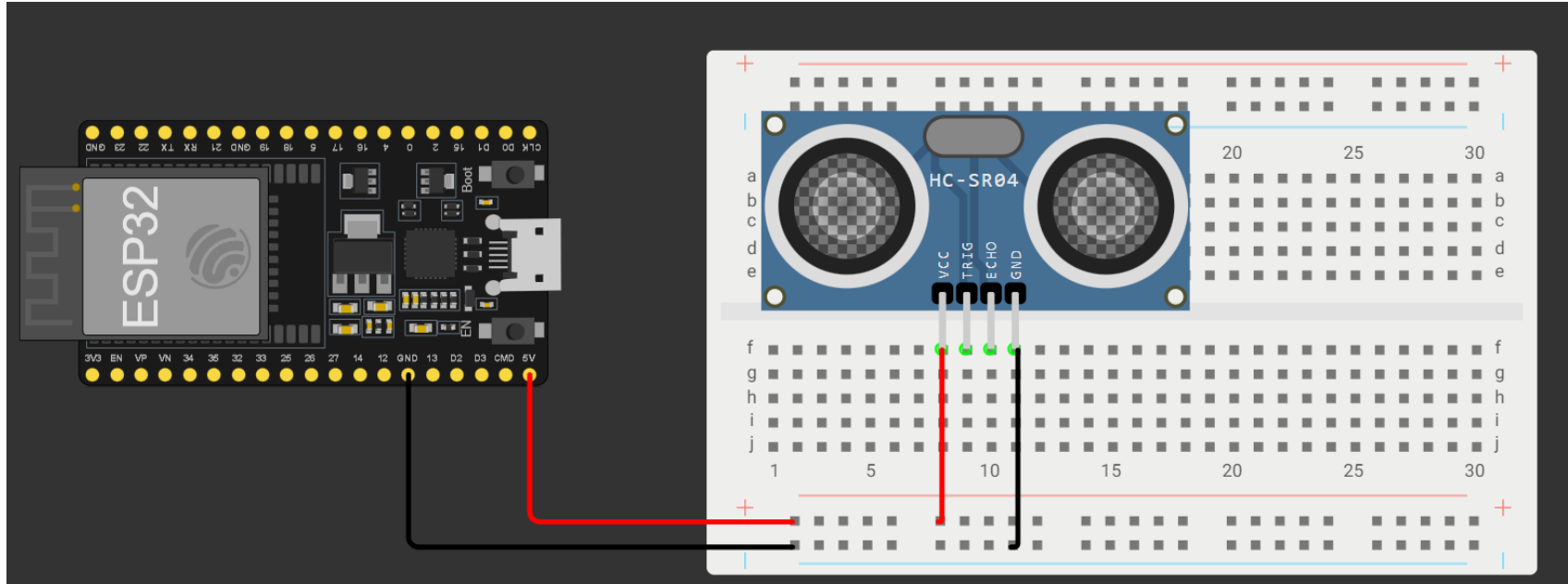
❖ Step1: Connecting the Ultrasonic Sensor to the Breadboard

- In this step, the HC-SR04 Ultrasonic Sensor is placed on the breadboard to make wiring easier.
Then, the ESP32 board is connected to the breadboard power rails:
- The 5V pin of the ESP32 is connected to the positive (red) power rail on the breadboard to provide power to the sensor.
- The GND pin of the ESP32 is connected to the negative (black) power rail on the breadboard to complete the ground connection.
- This prepares the breadboard to power the ultrasonic sensor in the next steps.



Ultrasonic Sensor

Step 2: Powering the Ultrasonic Sensor



Ultrasonic Sensor

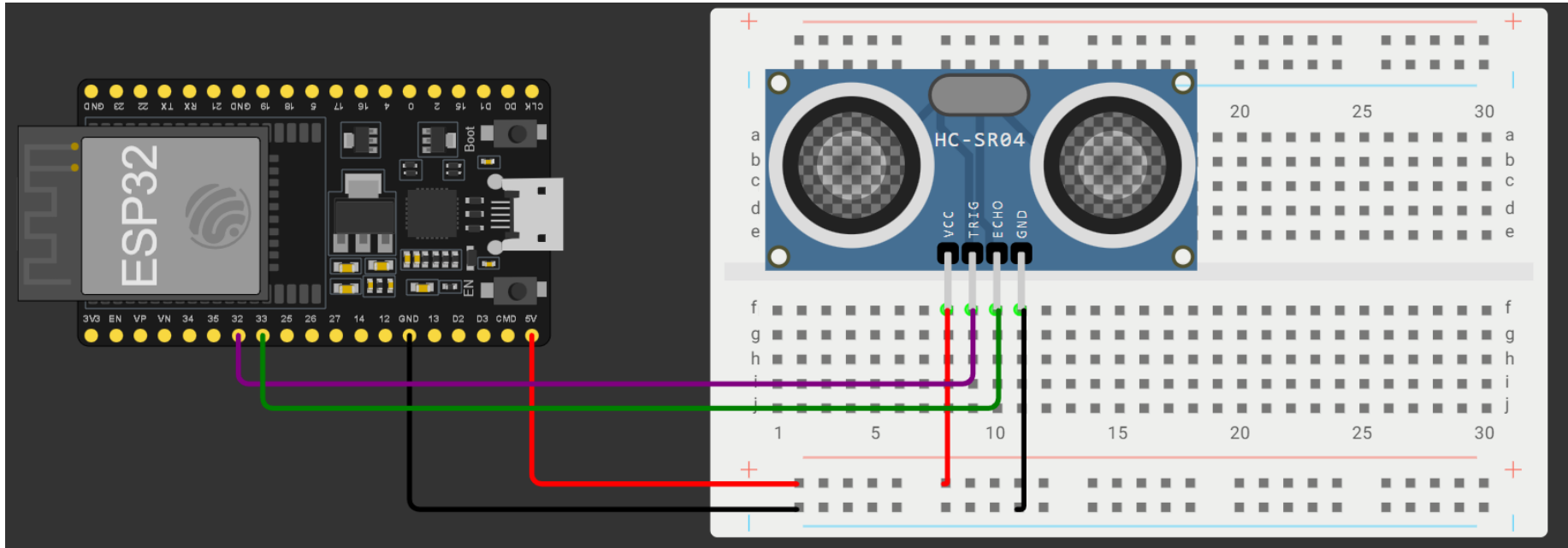
❖ Step 2: Powering the Ultrasonic Sensor

- In this step, the **HC-SR04 Ultrasonic Sensor** is connected to the power rails of the breadboard:
- The **VCC pin** of the sensor is connected to the **positive (red)** power rail on the breadboard, which provides **5V** from the ESP32.
- The **GND pin** of the sensor is connected to the **negative (black)** power rail on the breadboard, which is connected to **GND** on the ESP32.
- Now the sensor is powered and ready to be connected to the control pins (TRIG and ECHO) in the next step.



Ultrasonic Sensor

Step 3: Connecting the Control Pins (TRIG and ECHO)



Ultrasonic Sensor

❖ Step 3: Connecting the Control Pins (TRIG and ECHO)

- In this step, we connect the **control pins** of the ultrasonic sensor to the **ESP32**:
 - The **TRIG pin** of the sensor is connected to **GPIO 32** on the ESP32.
This pin is used to send the ultrasonic pulse.
 - The **ECHO pin** of the sensor is connected to **GPIO 33** on the ESP32.
This pin receives the reflected pulse to measure the distance.
- Now, the **HC-SR04 sensor** is fully connected to the **ESP32**, with power (5V and GND) and communication pins (TRIG and ECHO) properly wired.
- This setup allows the ESP32 to send a pulse and calculate the distance to nearby objects.

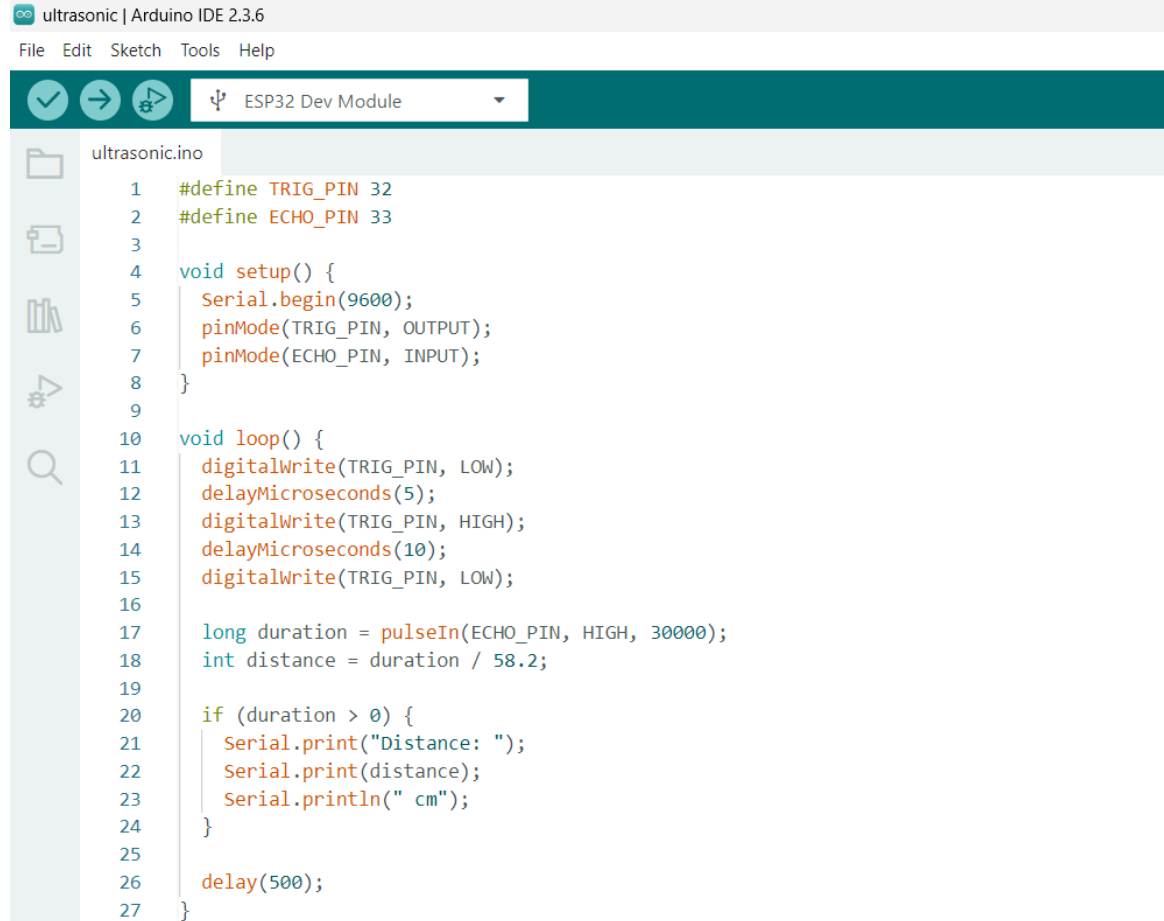


Ultrasonic Sensor



Ultrasonic Sensor

Code Explanation – Ultrasonic Sensor with ESP32 in Arduino IDE



The screenshot shows the Arduino IDE interface with the file 'ultrasonic.ino' open. The code is written in C++ and is designed to interface with an ultrasonic sensor on an ESP32. The code includes pin definitions, setup functions for serial communication and pin modes, and a loop function that sends a pulse to the trigger pin, measures the time until an echo is received, and prints the distance to the serial monitor.

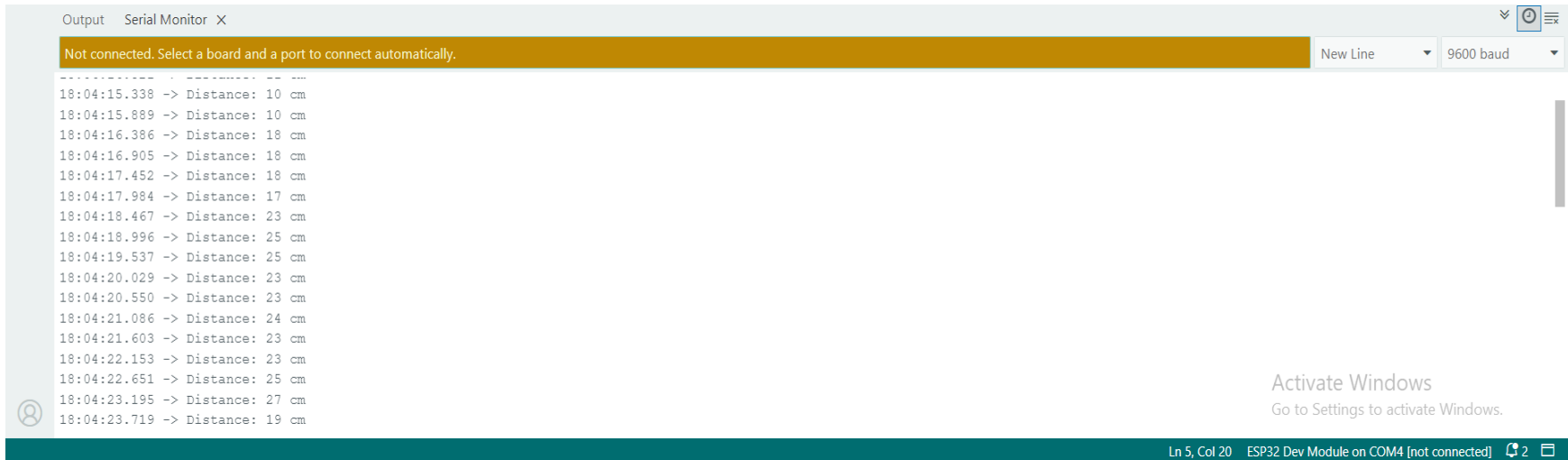
```
ultrasonic | Arduino IDE 2.3.6
File Edit Sketch Tools Help

ESP32 Dev Module

ultrasonic.ino
1  #define TRIG_PIN 32
2  #define ECHO_PIN 33
3
4  void setup() {
5      Serial.begin(9600);
6      pinMode(TRIG_PIN, OUTPUT);
7      pinMode(ECHO_PIN, INPUT);
8  }
9
10 void loop() {
11     digitalWrite(TRIG_PIN, LOW);
12     delayMicroseconds(5);
13     digitalWrite(TRIG_PIN, HIGH);
14     delayMicroseconds(10);
15     digitalWrite(TRIG_PIN, LOW);
16
17     long duration = pulseIn(ECHO_PIN, HIGH, 30000);
18     int distance = duration / 58.2;
19
20     if (duration > 0) {
21         Serial.print("Distance: ");
22         Serial.print(distance);
23         Serial.println(" cm");
24     }
25
26     delay(500);
27 }
```

Ultrasonic Sensor

Code Explanation – Ultrasonic Sensor with ESP32 in Arduino IDE



The screenshot shows the Arduino IDE Serial Monitor window. The title bar reads "Output Serial Monitor X". A yellow status bar at the top of the monitor area says "Not connected. Select a board and a port to connect automatically." To the right of this bar are two dropdown menus: "New Line" and "9600 baud". The main area of the monitor displays a series of timestamped distance readings in centimeters. The data is as follows:

Timestamp	Distance (cm)
18:04:15.338	10
18:04:15.889	10
18:04:16.386	18
18:04:16.905	18
18:04:17.452	18
18:04:17.984	17
18:04:18.467	23
18:04:18.996	25
18:04:19.537	25
18:04:20.029	23
18:04:20.550	23
18:04:21.086	24
18:04:21.603	23
18:04:22.153	23
18:04:22.651	25
18:04:23.195	27
18:04:23.719	19

At the bottom of the IDE window, a status bar shows "Ln 5, Col 20" and "ESP32 Dev Module on COM4 [not connected]". To the right of the status bar is a watermark for "Activate Windows" with the text "Go to Settings to activate Windows." and a large, faint Windows logo.

Step-by-step Explanation

1. Define Pins

```
#define TRIG_PIN 32  
#define ECHO_PIN 33
```

These lines assign the ESP32 pins connected to the sensor:

TRIG_PIN sends the sound wave.

ECHO_PIN receives the reflected signal.

2. Setup Function

```
void setup() {  
  Serial.begin(9600);  
  pinMode(TRIG_PIN, OUTPUT);  
  pinMode(ECHO_PIN, INPUT); }
```

Initializes the serial monitor at speed of 9600 to display

Distance readings

Sets TRIG as OUTPUT(to send pulses)

Sets ECHO as INPUT (to receive the echo signal)

3. Sending the Ultrasonic Pulse

```
digitalWrite(TRIG_PIN, LOW);  
delayMicroseconds(5);  
digitalWrite(TRIG_PIN, HIGH);  
delayMicroseconds(10);  
digitalWrite(TRIG_PIN, LOW);
```

This short sequence sends a **10-microsecond** pulse through the TRIG pin.

When the pulse is sent, the sensor emits an ultrasonic sound wave.

4. Measuring the Echo

```
long duration = pulseIn(ECHO_PIN, HIGH, 30000);  
int distance = duration / 58.2;
```

The pulseIn() function measures how long the echo pin stays high

The duration is divided by 58.2 to convert into distance in centimeters

Step-by-step Explanation

5. Displaying the Distance

```
if (duration > 0) {  
  Serial.print("Distance: ");  
  Serial.print(distance);  
  Serial.println(" cm");}
```

If the sensor receives an echo, the calculated distance is printed on the **Serial Monitor** in centimeters. If no echo is received (object too far or no obstacle), nothing is displayed.

6. Delay Between Readings

```
delay(500);
```

Adds a **0.5-second delay** before repeating the process this makes readings more stable and easier to read.

